

5장 API 연동과 인증 프로세스 구현

이번 장은 React 애플리케이션에서 외부 API와의 데이터 연동 및 사용자 인증 프로세스를 실제로 구현하는 과정을 다루며, `fetch`와 `axios`를 활용한 비동기 호출과 로딩·에러 처리, `Todo` 앱의 상태 관리와 로컬 저장, 그리고 로그인 API 연동과 JWT 기반 인증 상태 유지까지 학습함으로써 실무형 프론트엔드 애플리케이션 개발의 핵심 흐름을 완성한다.

5.1 fetch/axios API

`fetch`와 `axios`는 웹 애플리케이션에서 외부 API와 데이터를 주고받기 위한 핵심 도구로, 비동기 호출의 기본 구조와 차이를 이해하고 응답·에러·로딩 상태를 관리하며, 실제 외부 데이터 기반으로 화면을 구성하는 과정을 통해 실무적인 API 연동 능력을 익힌다.

(1) API 호출 방식 및 비동기 흐름

API 호출은 네트워크 요청이 완료될 때까지 기다리지 않고 이벤트 루프를 통해 비동기적으로 처리되며, `Promise` 또는 `async/await` 구문을 활용해 응답 결과를 순차적으로 제어함으로써 UI의 끊김 없는 데이터 연동을 가능하게 한다.

① **비동기 모델**: JS는 이벤트 루프 기반. 네트워크 요청은 비동기(브라우저/노드)로 처리되고, `Promise`가 완료되면 마이크로태스크 큐로 콜백이 실행되어 렌더링과 충돌을 피함.

② 제어 흐름

- 직렬(순차): `await a(); await b();`
- 병렬(동시): `await Promise.all([a(), b()])`

③ **취소**: `Fetch`는 `AbortController` 로 취소 가능. `axios`는 `signal` 옵션(또는 과거의 `CancelToken`)으로 취소 가능.

④ **오류전파**: `try/catch` 로 캡처, 재시도 정책(logic) 적용.

⑤ `async/await` + `AbortController` (`fetch`)

```
// src/examples/fetchWithAbort.js
export async function fetchWithAbort(url, timeout = 5000) {
  const controller = new AbortController();
  const signal = controller.signal;
```

```
// 타임아웃 자동 취소 (선택적)
const timer = setTimeout(() => controller.abort(), timeout);

try {
  const res = await fetch(url, { method: 'GET', signal });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const json = await res.json();
  return json;
} finally {
  clearTimeout(timer);
}
}
```

- `AbortController` 로 사용자가 취소하거나 타임아웃 시점에 요청을 중단한다.
- `response.ok` 검사로 HTTP 에러 상태를 명시적으로 처리해야 한다.
- `finally` 에서 타이머를 정리하여 리소스 누수 방지.

(2) fetch, axios의 사용법 비교

fetch는 브라우저 내장 API로 가볍고 표준화된 방식으로 요청을 처리하며, axios는 추가 기능이 풍부하고 JSON 자동 변환, 요청·응답 인터셉터, 타임아웃 설정 등 고급 기능을 제공해 프로젝트 성격에 따라 적합한 도구를 선택할 수 있다.

① fetch

- 내장 API, 추가 의존성 불필요
- `response.ok` 체크 후 `response.json()` 호출 필요
- 브라우저 호환성(구형) 주의(폴리필 필요)
- 취소: `AbortController`

② axios

- 자동으로 `response.data` 에 파싱된 JSON 반환

- 인터셉터(request/response), 기본 인스턴스 설정(baseURL, headers), timeout 설정 기능
- 에러 객체가 `error.response`, `error.request` 등으로 구분되어 유용
- 취소: `signal` 옵션(또는 older CancelToken)

③ fetch vs axios 사용

```
// fetch 예시 (GET)
async function fetchUsers(){
  const res = await fetch('https://jsonplaceholder.typicode.com/users');
  if(!res.ok) throw new Error('Fetch failed: ' + res.status);
  return await res.json();
}

// axios 예시 (GET) - axios 설치 필요
import axios from 'axios';
const api = axios.create({ baseURL: 'https://jsonplaceholder.typicode.com',
  timeout: 6000 });

api.interceptors.request.use(cfg => {
  // 토큰 첨부, 로깅 등
  // cfg.headers.Authorization = `Bearer ${token}`;
  return cfg;
});

async function axiosGetPosts(){
  const res = await api.get('/posts');
  return res.data; // axios는 이미 파싱된 data 반환
}
```

- `api.interceptors.request` 로 모든 요청에 공통 동작(예: 토큰 추가)을 삽입할 수 있다.
- `axios.create()` 로 인스턴스를 만들어 프로젝트 전역 설정(기본 헤더, 베이스 URL, 타임아웃 등)을 관리.

(3) 응답 처리·에러 처리·로딩 처리 구현

API 호출 과정에서 응답 데이터를 파싱해 상태로 저장하고 로딩 여부를 UI에 반영하며, 네트워크 실패나 서버 오류 발생 시 에러 메시지를 사용자 친화적으로 표시함으로써 안정적인 데이터 연동과 예외 처리 구조를 구현한다.

① 핵심 패턴

- 상태: `{ data, loading, error }`
- 사용자 친화적 에러 메시지: HTTP 상태 코드 → 인간 친화 텍스트
- 취소: 컴포넌트 언마운트 시 요청 취소
- 재시도: 필요 시 간단 재시도 로직(지수 백오프) 사용

② 공통 훅 `useApi` (로딩/에러/취소 처리)

```
// src/hooks/useApi.js
import { useState, useEffect } from 'react';

export default function useApi(apiFunc, deps = []) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
    setError(null);

    apiFunc()
      .then(res => { if(!cancelled) setData(res); })
      .catch(err => { if(!cancelled) setError(err.message || 'Unknown error'); })
      .finally(() => { if(!cancelled) setLoading(false); });

    return () => { cancelled = true; };
  }, deps);

  return { data, loading, error };
}
```

- `apiFunc` 는 내부에서 `fetch/axios` 호출을 수행해 파싱된 데이터를 반환하는 `async` 함수여야 함.
- `cancelled` 플래그는 마운트 해제 후 상태 업데이트를 방지. (`fetch`에선 `AbortController` 를 병행 권장)
- 컴포넌트에서는 `{ data, loading, error } = useApi(() => fetchUsers(), [])` 형태로 사용.

③ FetchDemo 컴포넌트 (useApi 사용)

```
// src/components/FetchDemo.js
import React from 'react';
import useApi from '../hooks/useApi';

const fetchUsers = async () => {
  const res = await fetch('https://jsonplaceholder.typicode.com/users');
  if (!res.ok) throw new Error('Network response was not ok: ' + res.status);
  return res.json();
}

export default function FetchDemo(){
  const { data, loading, error } = useApi(fetchUsers, []);
  if(loading) return <p>⌚ 로딩 중...</p>;
  if(error) return <p>❌ 에러: {error}</p>;
  return (
    <ul>
      {data.map(u => <li key={u.id}>{u.name} ({u.email})</li>)}
    </ul>
  );
}
```

- 로딩/에러/데이터 상태별 UI를 분기해서 표시한다.
- 에러 메시지는 내부에서 적절히 가공해 사용자에게 노출.

(4) 외부 데이터 기반 화면 구성 실습

외부 API로부터 받아온 JSON 데이터를 리스트나 카드 형태로 렌더링하고, 상태 관리와 조건부 렌더링을 적용하여 실시간으로 변화하는 데이터를 기반으로 완성도 높은 화면을 구성하는 실습을 진행한다.

① 실무 포인트

- **구조:** 상단 검색/필터 → 리스트/그리드 → 페이징/무한스크롤 → 상세 모달
- **UX 요소:** 스켈레톤 로더, 오류 바, 빈 상태(Empty) UI
- **성능:** 페이지네이션, 서버사이드 페이징, 캐시(메모리/localStorage), react-query/SWR 활용 권장
- **낙관적 업데이트(Optimistic UI):** POST/DELETE 후 UI를 먼저 갱신하고 서버응답으로 수정/복구

② AxiosDemo (카드형 UI, 상위 5개)

```
// src/components/AxiosDemo.js
import React from 'react';
import axios from 'axios';
import useApi from '../hooks/useApi';

const api = axios.create({ baseURL: 'https://jsonplaceholder.typicode.com',
  timeout: 6000 });

const fetchPosts = async () => {
  const res = await api.get('/posts');
  // 대량 데이터일 경우 서버에서 페이징을 요청하거나 slice 사용
  return res.data.slice(0, 5);
}

export default function AxiosDemo(){
  const { data, loading, error } = useApi(fetchPosts, []);
  if(loading) return <div className="skeleton">로딩중...</div>;
  if(error) return <div className="error">에러: {error}</div>;
  return (
    <div className="cards">
      {data.map(post => (
```

```

    <article key={post.id} className="card">
      <h3>{post.title}</h3>
      <p>{post.body}</p>
    </article>
  )}}
</div>
);
}

```

- 카드에 `key` 를 적절히 지정. 긴 리스트는 `virtualization` (react-window 등) 검토.
- 페이징 예: `page` 매개변수를 API에 전달하여 서버에서 잘라달라고 요청하거나 클라이언트에서 `slice` 사용.

(5) fetch/axios API 실습 프로젝트

fetch와 axios로 외부 API를 호출할 때의 비동기 흐름, 응답 파싱, 에러/로딩 처리, UI 구성 패턴을 이해하면 실무에서 안정적이고 사용자 친화적인 데이터 연동을 구현할 수 있습니다.

① 디렉터리 구조

```

ex01/
├── package.json
├── public/
│   └── index.html
└── src/
    ├── index.js
    ├── App.js
    ├── components/
    │   ├── FetchDemo.js    # fetch API 실습
    │   └── AxiosDemo.js    # axios 실습
    ├── hooks/
    │   └── useApi.js        # 공통 API 호출 커스텀 훅
    └── styles/
        └── App.css

```

② 실행 명령어

```
# React 프로젝트 생성
npx create-react-app ex01
cd ex01
```

```
# axios 설치 (fetch는 내장)
npm install axios
```

```
# 실행
npm start
```

③ src/index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./styles/App.css";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

④ src/App.js

```
import React from "react";
import FetchDemo from "./components/FetchDemo";
import AxiosDemo from "./components/AxiosDemo";

function App() {
  return (
```



```

<div className="app-container">
  <h1>5.1 fetch / axios API 실습</h1>
  <div className="demo-section">
    <h2>① fetch API</h2>
    <FetchDemo />
  </div>
  <div className="demo-section">
    <h2>② axios API</h2>
    <AxiosDemo />
  </div>
</div>
);
}

export default App;

```

⑤ `src/hooks/useApi.js` (공통 API 훅)

```

import { useState, useEffect } from "react";

function useApi(apiFunc, dependencies = []) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancel = false;
    setLoading(true);
    apiFunc()
      .then((res) => {
        if (!cancel) setData(res);
      })
      .catch((err) => {
        if (!cancel) setError(err.message);
      })
      .finally(() => {
        if (!cancel) setLoading(false);
      });
  }, dependencies);
}

```

```

    });
    return () => {
      cancel = true;
    };
  }, dependencies);

  return { data, loading, error };
}

export default useApi;

```

⑥ `src/components/FetchDemo.js`

```

import React from "react";
import useApi from "../hooks/useApi";

const FetchDemo = () => {
  const fetchUsers = async () => {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!response.ok) throw new Error("네트워크 오류 발생!");
    return response.json();
  };

  const { data, loading, error } = useApi(fetchUsers, []);

  if (loading) return <p>⌚ 로딩 중...</p>;
  if (error) return <p>❌ 에러: {error}</p>;

  return (
    <ul>
      {data.map((user) => (
        <li key={user.id}>
          {user.name} {user.email}
        </li>
      ))}
    </ul>
  );
}

```

```
);
};

export default FetchDemo;
```

⑦ src/components/AxiosDemo.js

```
import React from "react";
import axios from "axios";
import useApi from "../hooks/useApi";

const AxiosDemo = () => {
  const fetchPosts = async () => {
    const response = await axios.get("https://jsonplaceholder.typicode.com/posts");
    return response.data.slice(0, 5); // 앞 5개만 표시
  };

  const { data, loading, error } = useApi(fetchPosts, []);

  if (loading) return <p>⌚ 로딩 중...</p>;
  if (error) return <p>❌ 에러: {error}</p>;

  return (
    <div>
      {data.map((post) => (
        <div key={post.id} className="card">
          <h3>{post.title}</h3>
          <p>{post.body}</p>
        </div>
      ))}
    </div>
  );
};

export default AxiosDemo;
```

⑧ src/styles/App.css

```
.app-container {  
  max-width: 800px;  
  margin: 0 auto;  
  padding: 20px;  
  font-family: sans-serif;  
}  
  
.demo-section {  
  margin-bottom: 40px;  
}  
  
.card {  
  border: 1px solid #ccc;  
  border-radius: 10px;  
  padding: 15px;  
  margin-bottom: 15px;  
  background-color: #f9f9f9;  
}
```

- **fetch API** → 사용자 목록 가져오기
- **axios API** → 게시글 5개 가져오기
- 공통 `useApi` 훅으로 로딩/에러/응답 처리 구조를 통합 관리

5.2 Todo 애플리케이션 작성

Todo 애플리케이션은 상태 배열 관리, 항목 추가·삭제 로직, 조건부 렌더링을 통한 완료 여부 표시, 로컬 스토리지를 활용한 데이터 저장 기능을 통해 상태 관리와 데이터 지속성을 학습할 수 있는 대표적인 실습 예제다.

(1) 할 일 추가/삭제 로직 구현

사용자가 입력한 할 일을 배열 상태에 추가하고 필요 시 삭제 기능을 구현함으로써 입력 처리와 이벤트 기반 상태 변경의 기초 로직을 학습한다.

- `useState` 로 `todos` 배열 상태를 정의
- `setTodos([...todos, newTodo])` → 새로운 항목 추가
- `filter` 를 사용해 특정 id의 항목만 제외하여 삭제

아래 코드는 사용자가 입력한 할 일을 추가하고, 필요할 때 삭제하는 기본 동작을 구현합니다.

```
import React, { useState } from "react";

function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [input, setInput] = useState("");

  const addTodo = () => {
    setTodos([...todos, input]); // 새로운 항목 추가
    setInput("");
  };

  const removeTodo = (index) => {
    setTodos(todos.filter((_, i) => i !== index)); // index 기준 삭제
  };

  return (
    <div>
      <input value={input}
        onChange={(e) => setInput(e.target.value)}
        placeholder="할 일을 입력하세요"
      />
      <button onClick={addTodo}>추가</button>

      <ul>
        {todos.map((todo, index) => (
          <li key={index}>
            {todo} <button onClick={() => removeTodo(index)}>삭제</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

```

    </li>
  )))
</ul>
</div>
);
}

export default TodoApp;

```

- `useState([])` : 할 일 목록을 배열로 관리
- `setTodos([...todos, input])` : 기존 배열 복사 후 새 항목 추가
- `filter` : 삭제할 index 제외한 나머지 요소만 유지

(2) 상태 배열 관리 및 변경 처리

할 일 목록을 배열로 관리하며 `useState` 또는 `useReducer`를 활용해 항목의 추가, 삭제, 수정과 같은 변화를 불변성 원칙에 맞게 처리하여 React의 상태 관리 원리를 익힌다.

- React는 직접 배열을 수정하지 않고 **새 배열을 반환**해야 한다.
- `map`, `filter`, `concat` 등의 함수형 배열 메서드를 사용.
- 완료 여부 토글 시: `todos.map(todo => todo.id === id ? {...todo, completed: !todo.completed} : todo)`

아래 코드는 상태를 배열로 관리하며 불변성을 지켜 수정·삭제합니다.

```

const toggleTodo = (index) => {
  setTodos(
    todos.map((todo, i) =>
      i === index ? { ...todo, completed: !todo.completed } : todo
    )
  );
};

```

- `map` 을 사용하여 배열을 순회
- 클릭한 항목(`index`)만 `completed` 값을 반전

- `...todo` : 기존 속성을 유지하면서 `completed` 만 수정

(3) 조건부 렌더링을 통한 시각화

완료된 할 일과 미완료 항목을 조건부 렌더링으로 구분 표시하고, 스타일이나 아이콘을 활용해 시각적으로 명확히 표현함으로써 UI와 데이터 상태의 연결성을 경험한다.

- JSX 내 삼항 연산자(`?:`) 활용
- CSS 클래스 조건 적용: `className={todo.completed ? "done" : ""}`

아래 코드는 완료 여부에 따라 다른 스타일을 적용합니다.

```
<ul>
  {todos.map((todo, index) => (
    <li key={index}
      style={{ textDecoration: todo.completed ? "line-through" : "none" }}
      onClick={() => toggleTodo(index)}
    >
      {todo.text}
    </li>
  ))}
</ul>
```

- `todo.completed` 가 `true` 면 `line-through` 적용
- 항목 클릭 시 `toggleTodo` 실행 → 완료 여부 전환

(4) 간단한 저장 적용 - localStorage

할 일 데이터를 localStorage에 저장하고 컴포넌트 마운트 시 불러오는 기능을 구현하여 브라우저 새로고침 후에도 데이터가 유지되는 영속성을 실습한다.

- `localStorage.setItem("todos", JSON.stringify(todos))`
- `JSON.parse(localStorage.getItem("todos") || "[]")`

- `useEffect` 를 활용해 상태 변경 시 자동 저장

아래 코드는 새로고침해도 데이터가 유지되도록 저장합니다.

```
import { useEffect } from "react";

// 저장
useEffect(() => {
  localStorage.setItem("todos", JSON.stringify(todos));
}, [todos]);

// 불러오기
useEffect(() => {
  const saved = localStorage.getItem("todos");
  if (saved) {
    setTodos(JSON.parse(saved));
  }
}, []);
```

- `[todos]` 의존성 배열 → `todos` 가 변경될 때마다 `localStorage` 저장
- 마운트 시(`[]`) `localStorage`에서 값 불러오기

(5) Todo 애플리케이션 작성

Todo 애플리케이션은 입력한 할 일을 추가·삭제하고 완료 여부를 토글하며 조건부 렌더링을 통해 시각적으로 구분하고, `localStorage`를 활용해 데이터가 새로고침 후에도 유지되도록 구현함으로써 상태 관리, 이벤트 처리, 영속성 적용까지 React 핵심 개념을 종합적으로 학습할 수 있는 대표적인 실습 예제이다.

① 디렉터리 구조

```
ex02/
├── node_modules/
├── public/
└── src/
```



```

|   |— components/
|   |   |— TodoInput.js
|   |   |— TodoList.js
|   |   |— TodoItem.js
|   |— App.js
|   |— index.js
|   |— App.css
|   |— styles.css
|— package.json
|— README.md

```

② 설치 및 실행 명령

1. 프로젝트 생성

```
npx create-react-app ex02
```

```
cd ex02
```

2. 개발 서버 실행

```
npm start
```

③ src/App.js

```

import React, { useState, useEffect } from "react";
import TodoInput from "../components/TodoInput";
import TodoList from "../components/TodoList";
import "../App.css";

```

```

function App() {
  const [todos, setTodos] = useState(() => {
    // 초기 상태를 localStorage에서 불러오기
    const saved = localStorage.getItem("todos");
    return saved ? JSON.parse(saved) : [];
  });

```

```

  // todos 상태가 바뀔 때마다 localStorage에 저장
  useEffect(() => {

```

```

    localStorage.setItem("todos", JSON.stringify(todos));
  }, [todos]);

  const addTodo = (text) => {
    const newTodo = { id: Date.now(), text, completed: false };
    setTodos([...todos, newTodo]);
  };

  const toggleTodo = (id) => {
    setTodos(
      todos.map((todo) =>
        todo.id === id ? { ...todo, completed: !todo.completed } : todo
      )
    );
  };

  const deleteTodo = (id) => {
    setTodos(todos.filter((todo) => todo.id !== id));
  };

  return (
    <div className="App">
      <h1>  Todo List</h1>
      <TodoInput addTodo={addTodo} />
      <TodoList todos={todos} toggleTodo={toggleTodo} deleteTodo={delete
    Todo} />
    </div>
  );
}

export default App;

```

④ src/components/TodoInput.js

```

import React, { useState } from "react";

function TodoInput({ addTodo }) {

```

```

const [value, setValue] = useState("");

const handleSubmit = (e) => {
  e.preventDefault();
  if (!value.trim()) return;
  addTodo(value);
  setValue("");
};

return (
  <form onSubmit={handleSubmit} className="todo-input">
    <input type="text"
      placeholder="할 일을 입력하세요..."
      value={value}
      onChange={(e) => setValue(e.target.value)}
    />
    <button type="submit">추가</button>
  </form>
);
}

export default TodoInput;

```

⑤ `src/components/TodoList.js`

```

import React from "react";
import TodoItem from "../TodoItem";

function TodoList({ todos, toggleTodo, deleteTodo }) {
  if (todos.length === 0) return <p>할 일이 없습니다 ✨</p>;

  return (
    <ul className="todo-list">
      {todos.map((todo) => (
        <TodoItem key={todo.id}
          todo={todo}
          toggleTodo={toggleTodo}

```

```

        deleteTodo={deleteTodo}
      />
    )}}
  </ul>
);
}

export default TodoList;

```

⑥ src/components/TodoItem.js

```

import React from "react";

function TodoItem({ todo, toggleTodo, deleteTodo }) {
  return (
    <li className={`todo-item ${todo.completed ? "done" : ""}`}>
      <span onClick={() => toggleTodo(todo.id)}>{todo.text}</span>
      <button onClick={() => deleteTodo(todo.id)}>삭제</button>
    </li>
  );
}

export default TodoItem;

```

⑦ src/App.css

```

.App {
  max-width: 400px;
  margin: 50px auto;
  padding: 20px;
  border-radius: 12px;
  background: #f9f9f9;
  box-shadow: 0 2px 6px rgba(0,0,0,0.15);
  text-align: center;
  font-family: Arial, sans-serif;
}

```

```
h1 {
  color: #333;
  margin-bottom: 20px;
}

.todo-input {
  display: flex;
  gap: 10px;
  margin-bottom: 20px;
}

.todo-input input {
  flex: 1;
  padding: 8px;
}

.todo-input button {
  padding: 8px 12px;
  background: #0078d4;
  color: white;
  border: none;
  border-radius: 4px;
}

.todo-list {
  list-style: none;
  padding: 0;
}

.todo-item {
  display: flex;
  justify-content: space-between;
  background: white;
  padding: 10px;
  margin-bottom: 8px;
  border-radius: 6px;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
}
```

```

}

.todo-item.done span {
  text-decoration: line-through;
  color: gray;
}

```

5.3 로그인 프로세스 및 API 호출 구현

로그인 프로세스는 사용자 입력 기반 API 요청과 JWT 토큰 처리, 인증 여부에 따른 조건부 렌더링, 로그인 성공 시 홈 화면 전환을 통해 보안성과 사용자 경험을 동시에 고려한 인증 흐름을 구축하는 실무형 학습 과정이다.

(1) 로그인 요청 API 연동

아이디와 비밀번호 입력값을 기반으로 로그인 API를 호출하고, 서버 응답 결과에 따라 인증 여부를 판별해 로그인 성공·실패 시의 로직을 제어한다.

```

// LoginForm.js
import React, { useState } from "react";

function LoginForm({ onLogin }) {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault();
    // (API 요청)
    const response = await fetch("http://localhost:4000/api/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ username, password }),
    });

```

```

const data = await response.json();
if (response.ok) {
  onLogin(data.token); // 성공 시 토큰 전달
} else {
  alert("로그인 실패: " + data.message);
}
};

return (
  <form onSubmit={handleSubmit}>
    <input type="text"
      placeholder="아이디"
      value={username}
      onChange={(e) => setUsername(e.target.value)}
    />
    <input type="password"
      placeholder="비밀번호"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
    />
    <button type="submit">로그인</button>
  </form>
);
}

export default LoginForm;

```

사용자가 아이디/비밀번호를 입력하면 `fetch` 를 통해 서버 API(`/api/login`)로 POST 요청을 보냅니다. 성공 시 JWT 토큰을 받아 상위 컴포넌트로 전달합니다.

(2) JWT 토큰 처리 흐름

로그인 성공 시 서버로부터 전달받은 JWT 토큰을 로컬 스토리지나 쿠키에 저장하고, 이후 요청 시 인증 헤더에 포함시켜 안전하게 사용자 인증 상태를 유지한다.

```
// App.js
import React, { useState } from "react";
import LoginForm from "../components/LoginForm";
import Home from "../components/Home";

function App() {
  const [token, setToken] = useState(localStorage.getItem("token"));

  const handleLogin = (newToken) => {
    localStorage.setItem("token", newToken); // 저장
    setToken(newToken); // 상태 갱신
  };

  const handleLogout = () => {
    localStorage.removeItem("token"); // 제거
    setToken(null);
  };

  return (
    <div>
      {token ? (
        <Home onLogout={handleLogout} />
      ) : (
        <LoginForm onLogin={handleLogin} />
      )}
    </div>
  );
}

export default App;
```

로그인 성공 시 받은 JWT 토큰을 `localStorage`에 저장하여 새로고침 후에도 유지할 수 있습니다. 로그아웃 시에는 제거합니다.

(3) 인증 상태에 따른 조건부 렌더링

인증 여부에 따라 로그인 폼 또는 사용자 전용 콘텐츠를 조건부 렌더링으로 전환하여 사용자 경험을 향상시키고 불필요한 접근을 차단한다.

```
// Home.js
import React, { useEffect, useState } from "react";

function Home({ onLogout }) {
  const [profile, setProfile] = useState(null);

  useEffect(() => {
    const token = localStorage.getItem("token");
    fetch("http://localhost:4000/api/profile", {
      headers: { Authorization: `Bearer ${token}` },
    })
      .then((res) => res.json())
      .then((data) => setProfile(data));
  }, []);

  return (
    <div>
      <h1>홈 화면</h1>
      {profile ? (
        <p>환영합니다, {profile.username}님 🎉</p>
      ) : (
        <p>사용자 정보를 불러오는 중...</p>
      )}
      <button onClick={onLogout}>로그아웃</button>
    </div>
  );
}

export default Home;
```

토큰이 있으면 `Home` 화면을 보여주고, 없으면 `LoginForm` 을 보여줍니다. 또, 토큰을 이용해 프로필 API를 호출하여 인증된 사용자만 접근할 수 있도록 조건부 렌더링합니다.

(4) 로그인 홈 화면 전환 구현

로그인 성공 시 홈 화면 또는 대시보드로 리다이렉트되도록 구현하여 인증 후 사용자 흐름을 자연스럽게 이어가는 전환 과정을 학습한다.

```
// App.js (추가 - React Router 활용)
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";

function App() {
  const [token, setToken] = useState(localStorage.getItem("token"));

  const handleLogin = (newToken) => {
    localStorage.setItem("token", newToken);
    setToken(newToken);
  };

  return (
    <BrowserRouter>
      <Routes>
        <Route path="/"
          element={token ? <Home /> : <Navigate to="/login" />}
        />
        <Route path="/login" element={<LoginForm onLogin={handleLogin} />} />
      </Routes>
    </BrowserRouter>
  );
}
```

로그인 성공 시 `/` 경로(홈 화면)로 자동 전환되도록 `Navigate` 를 사용합니다. 로그인이 안 되어 있으면 `/login` 으로 강제 이동시켜 보안 흐름을 유지합니다.

(5) 로그인 기능 구현 실습

아래 실습은 로컬 `public/member.json` 을 이용한 모의 로그인, JWT(학습용 페이크 토큰) 생성·저장, Context 기반 전역 인증 상태, 보호 라우트, 로그인 후 홈 리다이렉트, 에러·로딩 처리

등을 포함합니다. 코드를 복사해 `create-react-app` 으로 바로 실행할 수 있도록 구성했습니다.

① 디렉터리 구조

```
ex03/
├── package.json
├── public/
│   ├── index.html
│   └── member.json
└── src/
    ├── index.js
    ├── App.js
    ├── App.css
    ├── utils/
    │   ├── api.js
    │   └── jwt.js
    ├── context/
    │   └── AuthContext.js
    ├── components/
    │   ├── Header.js
    │   ├── LoginForm.js
    │   └── ProtectedRoute.js
    └── pages/
        ├── LoginPage.js
        └── HomePage.js
```

② 설치 및 실행 명령

1) 프로젝트 생성 (바로 시작하려면 아래 대로)

```
npx create-react-app ex03
```

```
cd ex03
```

2) 라우터 설치

```
npm install react-router-dom@6
```

3) (선택) styled-components 등 추가 패키지 필요시 설치

```
# npm install styled-components
```

```
# 4) 코드를 src/ 및 public/에 붙여넣은 뒤 실행  
npm start
```

③ public/member.json (모의 사용자 데이터)

```
[  
  { "id": 1, "username": "alice", "password": "alice1234", "name": "Alice Kim", "role": "user" },  
  { "id": 2, "username": "bob", "password": "bob1234", "name": "Bob Lee", "role": "admin" }  
]
```

④ src/utils/jwt.js (학습용 페이크 JWT 생성/파싱)

```
// src/utils/jwt.js  
// 간단한 Base64(학습용)으로 토큰 생성/파싱 — 실제 서비스에서는 서버 서명 검증 필요  
export function createFakeJWT(payload = {}, expiresInSec = 60 * 60 * 24)  
{  
  const hdr = { alg: "HS256", typ: "JWT" };  
  const now = Math.floor(Date.now() / 1000);  
  const body = { ...payload, iat: now, exp: now + expiresInSec };  
  const b64 = (o) => btoa(unescape(encodeURIComponent(JSON.stringify(o))));  
  return `${b64(hdr)}.${b64(body)}.signature`;  
}  
  
export function parseFakeJWT(token) {  
  try {  
    const parts = token.split(".");  
    if (parts.length < 2) return null;  
    const payload = JSON.parse(decodeURIComponent(escape(atob(parts[1]))));  
    return payload;  
  }  
}
```

```

    } catch (e) {
      return null;
    }
  }

export function isExpired(token) {
  const p = parseFakeJWT(token);
  if (!p || !p.exp) return true;
  return Math.floor(Date.now() / 1000) >= p.exp;
}

```

⑤ src/utils/api.js (모의 로그인 함수 — member.json 조회)

```

// src/utils/api.js
import { createFakeJWT } from "../jwt";

export async function mockLogin({ username, password }) {
  // member.json은 public 폴더에 있어 /member.json 으로 접근 가능
  const res = await fetch("/member.json");
  if (!res.ok) throw new Error("사용자 데이터 로드 실패");
  const users = await res.json();
  // 간단한 인증 (학습용) — plaintext 비밀번호 비교
  const user = users.find((u) => u.username === username && u.password === password);
  // 인위적 지연 (UX용)
  await new Promise((r) => setTimeout(r, 600));
  if (!user) {
    const err = new Error("아이디 또는 비밀번호가 올바르지 않습니다.");
    err.code = "INVALID_CREDENTIALS";
    throw err;
  }
  const token = createFakeJWT({ sub: user.id, username: user.username, name: user.name, role: user.role }, 60 * 60 * 24);
  return { token, user: { id: user.id, username: user.username, name: user.name, role: user.role } };
}

```

⑥ src/context/AuthContext.js (전역 인증 상태)

```
// src/context/AuthContext.js
import React, { createContext, useContext, useEffect, useMemo, useState }
from "react";
import { parseFakeJWT, isExpired } from "../utils/jwt";

const AuthContext = createContext(null);
const TOKEN_KEY = "ex03_token";

export function AuthProvider({ children }) {
  const [token, setToken] = useState(() => localStorage.getItem(TOKEN_KEY));
  const [user, setUser] = useState(() => {
    const t = localStorage.getItem(TOKEN_KEY);
    if (!t) return null;
    if (isExpired(t)) {
      localStorage.removeItem(TOKEN_KEY);
      return null;
    }
    const p = parseFakeJWT(t);
    return p ? { id: p.sub, username: p.username, name: p.name, role: p.role } : null;
  });

  useEffect(() => {
    if (!token) {
      setUser(null);
      localStorage.removeItem(TOKEN_KEY);
      return;
    }
    if (isExpired(token)) {
      setToken(null);
      setUser(null);
      localStorage.removeItem(TOKEN_KEY);
      return;
    }
  });
}
```

```

const p = parseFakeJWT(token);
setUser(p ? { id: p.sub, username: p.username, name: p.name, role: p.role } : null);
}, [token]);

const login = ({ token, user }) => {
  localStorage.setItem(TOKEN_KEY, token);
  setToken(token);
  setUser(user);
};

const logout = () => {
  localStorage.removeItem(TOKEN_KEY);
  setToken(null);
  setUser(null);
};

const value = useMemo(() => ({ token, user, isAuthenticated: !!token && !!user, login, logout }), [token, user]);

return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used within AuthProvider");
  return ctx;
}

```

⑦ src/components/ProtectedRoute.js (인증 보호 라우트)

```

// src/components/ProtectedRoute.js
import React from "react";
import { Navigate, Outlet } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

```

```
export default function ProtectedRoute() {
  const { isAuthenticated } = useAuth();
  return isAuthenticated ? <Outlet /> : <Navigate to="/login" replace />;
}
```

⑧ src/components/LoginForm.js (폼 컴포넌트 — 로딩/에러 처리 포함)

```
// src/components/LoginForm.js
import React, { useReducer, useState } from "react";
import { mockLogin } from "../utils/api";

const init = { username: "", password: "" };
function reducer(state, action) {
  switch (action.type) {
    case "SET": return { ...state, [action.key]: action.value };
    case "RESET": return init;
    default: return state;
  }
}

export default function LoginForm({ onSuccess }) {
  const [state, dispatch] = useReducer(reducer, init);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const submit = async (e) => {
    e.preventDefault();
    setError(null);
    setLoading(true);
    try {
      const result = await mockLogin({ username: state.username.trim(), password: state.password });
      onSuccess(result);
      dispatch({ type: "RESET" });
    } catch (err) {
      setError(err.message || "로그인 실패");
    } finally {

```



```

    setLoading(false);
  }
};

return (
  <form onSubmit={submit} style={{ maxWidth: 420 }}>
    <div style={{ marginBottom: 8 }}>
      <label>아이디</label>
      <input value={state.username} onChange={(e) => dispatch({ type: "SET", key: "username", value: e.target.value })} required />
    </div>
    <div style={{ marginBottom: 8 }}>
      <label>비밀번호</label>
      <input type="password" value={state.password} onChange={(e) => dispatch({ type: "SET", key: "password", value: e.target.value })} required />
    </div>
    {error && <div style={{ color: "crimson", marginBottom: 8 }}>{error}</div>}
    <button type="submit" disabled={loading}>{loading ? "로그인 중..." : "로그인"}</button>
  </form>
);
}

```

⑨ src/components/Header.js

```

// src/components/Header.js
import React from "react";
import { Link } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

export default function Header() {
  const { isAuthenticated, user, logout } = useAuth();
  return (
    <header style={{ display: "flex", justifyContent: "space-between", padding: 12, borderBottom: "1px solid #eee" }}>
      <div><Link to="/">EX03</Link></div>
    </header>
  );
}

```

```

    <div>
      {isAuth ? (
        <>
          <span style={{ marginRight: 12 }}>{user?.name} ({user?.role})</span>
          <button onClick={logout}>로그아웃</button>
        </>
      ) : (
        <Link to="/login"><button>로그인</button></Link>
      )}
    </div>
  </header>
);
}

```

⑩ src/pages/LoginPage.js

```

// src/pages/LoginPage.js
import React from "react";
import LoginForm from "../components/LoginForm";
import { useAuth } from "../context/AuthContext";
import { useNavigate } from "react-router-dom";

export default function LoginPage() {
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSuccess = ({ token, user }) => {
    login({ token, user });
    // 로그인 후 홈으로 이동
    navigate("/home", { replace: true });
  };

  return (
    <main style={{ padding: 24 }}>
      <h2>로그인</h2>
      <LoginForm onSuccess={handleSuccess} />
    </main>
  );
}

```

```

    </main>
  );
}

```

⑪ src/pages/HomePage.js

```

// src/pages/HomePage.js
import React from "react";
import { useAuth } from "../context/AuthContext";

export default function HomePage() {
  const { user } = useAuth();
  return (
    <main style={{ padding: 24 }}>
      <h2>홈</h2>
      <p>환영합니다, {user?.name} 님!</p>
      <p>이곳은 보호된 홈 화면입니다.</p>
    </main>
  );
}

```

⑫ src/App.js (라우팅 + 레이아웃)

```

// src/App.js
import React from "react";
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import { AuthProvider } from "../context/AuthContext";
import Header from "../components/Header";
import LoginPage from "../pages/LoginPage";
import HomePage from "../pages/HomePage";
import ProtectedRoute from "../components/ProtectedRoute";
import "../App.css";

function App() {
  return (

```

```

    <AuthProvider>
      <BrowserRouter>
        <Header />
        <Routes>
          <Route path="/" element={<Navigate to="/home" replace />} />
          <Route path="/login" element={<LoginPage />} />
          <Route element={<ProtectedRoute />}>
            <Route path="/home" element={<HomePage />} />
          </Route>
          <Route path="*" element={<Navigate to="/home" replace />} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  );
}

export default App;

```

⑬ src/index.js

```

import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);

```

⑭ src/App.css

```

body { font-family: Arial, Helvetica, sans-serif; margin:0; background:#f7f8fa; color:#222; }
a { text-decoration:none; color:inherit; }
button { cursor:pointer; padding:6px 10px; border-radius:6px; border:1px solid #ddd; background:#fff; }
input { padding:8px; border:1px solid #ccc; border-radius:6px; width:100%;

```

```
box-sizing: border-box; }  
main { padding: 20px; max-width: 900px; margin: 0 auto; }
```

⑮ 동작 흐름

1. `/login` 에서 아이디/비밀번호 입력 → `mockLogin()` 호출 (`member.json`에서 조회)
2. 성공 시 `{ token, user }` 반환 → `AuthContext.login()` 으로 token/user 저장 (`localStorage`)
3. `AuthProvider` 가 전역 `isAuth` 상태를 제공 → `ProtectedRoute` 가 보호 라우트로 동작
4. 인증 상태일 때 `/home` 에 접근 가능, 비인증시 `/login` 로 리다이렉트
5. 로그아웃 시 `localStorage` 제거 및 인증 상태 초기화